

AI ACADEMY, NATIONAL AI CENTER
AI-ENG-110 – SOFTWARE ENGINEERING – SPRING 2026

Topic 1 – Smart Lost & Found

AI-powered photo-to-photo item matching

Team Avaz / Kazim / Gulnar

Avaz • Kazim • Gulnar

Final Project Defense – 22 May 2026

<https://github.com/AvazAsgarov/smart-lost-and-found/tree/dev>

The problem in one sentence

What the system does

A user uploads a photo of an item they lost or found; the service uses a vision-language model to describe it, embeds the description, and returns the top- k most similar items from the opposite pool together with a short, human-readable reason for each match.

Inputs

- ▶ Image (PNG / JPEG, ≤ 5 MB)
- ▶ Optional free-text description
- ▶ Status (lost or found)

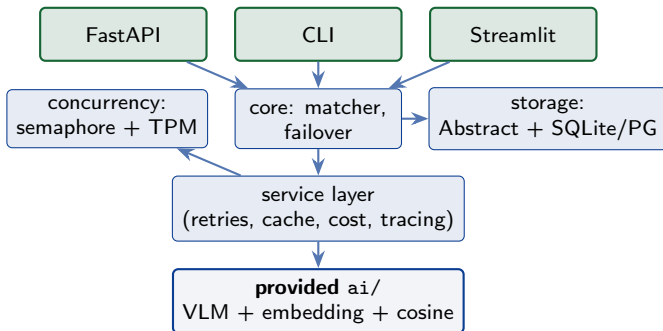
Outputs

- ▶ Ranked MatchRecord list with cosine score and reason
- ▶ Persisted item row (id, VLM description, embedding)
- ▶ Aggregated AI cost report on demand

Entry points: REST API (7 routes), argparse CLI (5 commands), Streamlit Web UI (5 pages).

Architecture

Three entry points → one service layer → swappable storage. The boundary between the *provided ai/* package and *our* SE layer is shown explicitly.



Swap providers = zero code changes. Swap SQLite → PostgreSQL = one env var.

Three design decisions we can defend

1. `asyncio + Semaphore(10)` **over threads.**

The bottleneck is network RTT, not CPU. FastAPI is already async, so threading would have meant two concurrency models in one project.

2. `AbstractRepository` **ABC over a Protocol.**

The ABC carries a shared `save_image()` implementation that both backends reuse; `TypeError` at `initialize()` time catches incomplete subclasses immediately.

3. **UUID image filenames with path-escape guard.**

The original filename never reaches the filesystem. Even `../../etc/passwd.png` resolves safely inside `IMAGES_DIR` or raises.

Expect to be asked

“Why not threads / Protocol / blob storage?” Each of us has a 15-second answer ready for every choice above.

Concurrency: the benchmark

Workload. Register 12 items end-to-end against an in-memory SQLite, fresh embedding cache, offline FakeVLM. Command: `python scripts/bench.py -n 12`.

	Sequential	Concurrent (sem=10)
Wall-clock (12 items)	0.18 s	0.04 s
Wall-clock (6 items)	0.05 s	0.01 s
Speedup	1.0×	4.5×
Provider 429s	0	0 (pre-emptive TPM budget)

Bottleneck named: the SQLite single-writer constraint on the INSERT path, plus the provider's tokens-per-minute ceiling.

Why not $N\times$: writes serialise on one connection; against a real provider we'd also be capped by the rate limiter we run *before* the call.

Numbers reproduce from the README – the exact command line is ready.

Robustness: one concrete failure mode

Scenario. Primary VLM (OpenAI) returns `ProviderError` on every retry; secondary VLM (Gemini) succeeds.

How we surfaced it. Failover unit test that monkey-patches `primary.describe` to raise; secondary returns a valid JSON.

What the system does:

- ▶ `FailoverVLM.describe` catches `ProviderError`, walks to the next ranked provider, and returns its response unchanged.
- ▶ Tenacity wraps the wrapper (3 attempts, exp backoff 1–10 s); failover is tried once per attempt, not per provider.
- ▶ Two log lines tell the whole story: `failover.tried provider=openai (WARN)` then `failover.fell_back new_active=gemini (INFO)`.
- ▶ User sees a clean HTTP 201 – no degraded fields, no retry-after header, latency +1 provider RTT.

The bug we caught early

First design retried the primary three times before triggering failover. The call-count

Live demo (or scripted demo)

What we will run

`scripts/demo.py` – registers 5 lost + 5 found items with deterministic fakes, runs a match against the lost pool, and prints the cost report. No API keys required.

Demo commands (work locally and inside Docker):

```
$ python scripts/demo.py # native
$ docker run --rm --env-file .env smart-lost-and-found \
    python scripts/demo.py # in container
```

What appears on screen:

- ▶ Per-item registration log lines
- ▶ Top-3 matches with score + reason
- ▶ Aggregated cost report (provider, tokens, \$)

If the network is slow we use the pre-recorded screencast.

Backup plan:

- ▶ Pre-recorded screencast (docs/demo.mp4)
- ▶ Screenshots in docs/screenshots/

Testing

Coverage & shape

- ▶ Total: **164 tests passing**, 0 failed
- ▶ src/ coverage: **71%**
- ▶ Provided ai/ smoke tests: **passing**
- ▶ Same 164 tests pass **inside the Docker image**
- ▶ All tests offline; FakeVLM / FakeEmbedder

Three tests worth calling out

- ▶ `test_smoke_e2e`: full happy path register → match
- ▶ `test_pipeline::returns_exception`: one task raises, others succeed
- ▶ `test_failover::switches_to_second_provider`: error transparently recovered

Tooling

`pytest + pytest-asyncio + pytest-cov`; **ruff clean**, **mypy clean** on all 19 source files. Coverage gate enforced in GitHub Actions CI.

Limitations & what we'd do next

Where the system is brittle today

- ▶ In-process embedding cache only – no sharing across workers
- ▶ Local-filesystem image storage – single-machine
- ▶ CostMeter records spend but does not enforce a hard ceiling
- ▶ PostgreSQL backend passes contract tests, not yet load-tested

Given another week we would

1. Move the embedding cache to Redis, keyed on (model, text)
2. Push images to S3 + presigned URLs; keep only the key in the DB
3. Add SSE / WebSocket progress reporting for batch endpoints
4. Wire a hard budget ceiling into TokenBudget

Honest worst design decision: the failover configuration was a free function instead of a constructor argument; we would refactor to constructor-injected dependencies.

Team contributions

Member	Primary ownership	Share
Avaz	Storage layer (SQLite + PostgreSQL), FastAPI endpoints, concurrency pipeline, matcher, Docker, release tag	~35%
Kazim	AI service wrappers, multi-provider failover, cost meter, token rate limiter, OpenTelemetry tracing	~33%
Gulnar	Pytest suite (164 tests), argparse CLI, Streamlit UI, GitHub Actions CI, benchmarks, e2e smoke	~32%

We can all answer

Pick any file in the repo and any of us will walk through it. The review queue in our PR template required at least one teammate to pull and run the verification commands before merge.

AI tool use: Claude Code was used as a development assistant for implementation support, debugging, refactoring, and documentation. Every change was reviewed and adapted by the team before merge.

Questions?

Team Avaz / Kazim / Gulnar

<https://github.com/AvazAsgarov/smart-lost-and-found/tree/dev>

Tag: v1.0-final • 164/164 tests • 71% coverage

"Pick any file – we'll walk through it."